FLOCK Project Report

BSD-style File Locking in mCertiKOS

Course:	CSE 4502 — Operating System Lab
Project:	File Locking (FLOCK) in mCertiKOS
Date:	March 29, 2026
OS Target:	mCertiKOS (teaching OS)

Team Members

Name	Student ID
Sanjana Afreen	220042106
Nishat Tasneem	220042110
Nafisa Tasnim	220042114
Nishat Tasnim	220042124
Mrittika Jahan	220042150

Department of Computer Science and Engineering

Contents

1 Executive Summary **3**

2 Base Project Description **3**
2.1 What is mCertiKOS? . . . 3
2.2 Provided Base Functionality . . . 3
2.3 What Was Missing . . . 4

3 Project Objectives **4**

4 Locking Model and Semantics **4**
4.1 Supported Operations . . . 4
4.2 Shared Lock (`LOCK_SH`) . . . 4
4.3 Exclusive Lock (`LOCK_EX`) . . . 5
4.4 Unlock (`LOCK_UN`) . . . 5
4.5 Non-Blocking Mode (`LOCK_NB`) . . . 5

5 Modules and Features Added **5**
5.1 Core Flock Engine (`kern/flock/`) . . . 5
5.2 Writer Priority Strategy . . . 6
5.3 Upgrade and Downgrade Support . . . 6
5.4 File-System Integration (Modified Files) . . . 6
5.5 Syscall Plumbing (Modified Files) . . . 7
5.6 Test Binaries (New Files) . . . 8

6 System Architecture **8**
6.1 Layered Syscall Pipeline . . . 8
6.2 Call Flow Description . . . 8
6.3 Per-Inode Lock State . . . 9
6.4 Per-Descriptor Ownership Tracking . . . 9
6.5 Automatic Cleanup on Close . . . 9
6.6 Multi-Process Test Coordination Protocol . . . 9

7 Technologies, Frameworks, and Libraries Used **10**

8 How Each Module Enhances the Project **11**
8.1 `kern/flock/` — Core Lock Engine . . . 11
8.2 Inode-Level `flock_t` — Correct Scope for Lock State . . . 11
8.3 File Descriptor `holding_flock` — Safe Resource Management . . . 11
8.4 Syscall Plumbing — Usable from User Space . . . 11
8.5 `flocktest` and `flockchild` — Confidence in Correctness . . . 11

9 Testing and Validation **11**
9.1 Test Suite Overview . . . 11
9.2 Test Category Breakdown . . . 13
9.3 How to Run . . . 13

10 Problems Encountered and How They Were Solved **13**
10.1 Consistent Coordination Across Multiple File Descriptors 13
10.2 Avoiding Lock Leaks 14
10.3 Writer Starvation Under High Reader Load 14
10.4 Multi-Process Test Coordination Without Shared Memory 14
10.5 Ensuring Inode Lock State Is Never Stale 14

11 Learning Outcomes **14**

12 Conclusion **15**

A File Inventory **16**

1. Executive Summary

This report documents the design, implementation, and validation of BSD-style file locking (`flock`) in the **mCertiKOS** operating system — a teaching OS used for systems programming education.

The project adds a complete file locking subsystem that allows multiple processes to safely coordinate access to shared files. The implementation introduces **shared** and **exclusive** lock semantics, supports both **blocking** and **non-blocking** acquisition modes, and integrates seamlessly from the user-space API down to kernel-level inode state.

Key contributions:

- A new core lock engine (`kern/flock/`) with shared, exclusive, and upgrade/downgrade logic.
- Integration into the kernel file-system at both the inode and file descriptor layers.
- Full syscall plumbing from user space through the kernel trap handler.
- A comprehensive 16-test validation suite covering single-process and multi-process scenarios.

2. Base Project Description

2.1 What is mCertiKOS?

mCertiKOS is a teaching operating system derived from the certified CertiKOS OS developed at Yale University. It provides a minimal but functional kernel with essential OS features including process management, virtual memory, inter-process communication, and a simple file system. It runs on x86 hardware emulated via QEMU and is used at the undergraduate level for hands-on OS programming assignments.

2.2 Provided Base Functionality

The base mCertiKOS system provided the following components which this project built upon:

- A working xv6-inspired file system with inodes, directories, file descriptors, and basic I/O system calls (`open`, `read`, `write`, `close`).
- Kernel synchronization primitives: spinlocks, `thread_sleep`, and `thread_wakeup` for blocking and waking processes.
- A syscall dispatch architecture (`TDispatch.c`) that routes syscall numbers to registered kernel handlers.
- A process/thread model with a thread control block (TCB) holding per-process open file descriptor tables.
- A user-space standard library (libc equivalent) with stdio, string, and syscall wrapper utilities.
- A basic shell for running user-space programs inside the OS.

2.3 What Was Missing

The base system had no mechanism for processes to coordinate access to shared files. Concurrent reads and writes to the same file could race, leading to data corruption or inconsistent reads. The `flock` system call — standard on all POSIX and BSD systems — was entirely absent: no kernel data structure, no syscall number, no handler, and no user-space API existed for it.

3. Project Objectives

The key objectives of the project were:

1. Implement file locking semantics compatible with POSIX `flock(2)` behavior.
2. Support concurrent read access through shared locks (`LOCK_SH`).
3. Ensure mutually exclusive write access through exclusive locks (`LOCK_EX`).
4. Provide a non-blocking acquisition mode (`LOCK_NB`) for immediate failure on contention.
5. Integrate locking into the kernel file-system path (inode and file descriptor layers).
6. Prevent lock leaks via automatic cleanup on file descriptor close.
7. Validate all functionality with a comprehensive user-space test suite.

4. Locking Model and Semantics

4.1 Supported Operations

The implementation exposes four lock operations, matching standard BSD `flock` semantics:

`Flag`	**Description**
`LOCK_SH`	Acquire a shared lock. Multiple holders allowed simultaneously.
`LOCK_EX`	Acquire an exclusive lock. Only one holder; all others blocked.
`LOCK_UN`	Release the lock currently held by this file descriptor.
`LOCK_NB`	Non-blocking modifier. Returns failure immediately if lock cannot be granted.

Table 1: Supported `flock` operations

4.2 Shared Lock (`LOCK_SH`)

A shared lock permits multiple concurrent holders. This models the *readers* role: any number of processes may hold a shared lock at the same time, enabling safe concurrent reads. A shared lock is incompatible with an exclusive lock — if an exclusive lock is currently held, a new shared request will either block or fail depending on `LOCK_NB`.

4.3 Exclusive Lock (`LOCK_EX`)

An exclusive lock allows only a single holder and blocks all other shared or exclusive requests. This models the *writer* role. No new readers or writers may proceed until the exclusive holder releases the lock.

4.4 Unlock (`LOCK_UN`)

Unlock releases the lock currently associated with the requesting file descriptor. After release, blocked waiters are woken up and may re-compete for the lock.

4.5 Non-Blocking Mode (`LOCK_NB`)

When `LOCK_NB` is combined with `LOCK_SH` or `LOCK_EX`, the kernel does not put the calling thread to sleep on contention. Instead, the call returns immediately with an error, allowing the caller to handle contention without blocking.

5. Modules and Features Added

The FLOCK project introduced entirely new modules and modified several existing kernel subsystems. This section describes each addition in detail.

5.1 Core Flock Engine (`kern/flock/`)

This is the primary new module added to the kernel, living in a new directory `kern/flock/`.

File	Responsibility
`flock.h`	Lock state types (`FLOCK_INACTIVE`, `FLOCK_SHARED`, `FLOCK_EXCLUSIVE`); `flock_t` structure; user-facing constants (`LOCK_SH`, `LOCK_EX`, `LOCK_UN`, `LOCK_NB`).
`flock.c`	`flock_init`: initialize lock state and wait queues. `flock_acquire`: shared/exclusive acquisition with blocking and non-blocking paths. `flock_release`: release logic and wake blocked threads.
`export.h`	Public API surface: `flock_init`, `flock_acquire`, `flock_release`.

Table 2: Core flock module files

The central `flock_t` data structure:

```
typedef struct flock_t {
    spinlock_t lock;            // Protects the flock structure
    enum flock_state state;     // FLOCK_INACTIVE / SHARED /
   EXCLUSIVE
    int shared_holders;         // Active shared lock holders
    int waiting_shared;         // Queued shared lock requests
    int waiting_exclusive;      // Queued exclusive lock
   requests
```

```
    struct {
        unsigned int pids[NUM_IDS];
        unsigned int len;
        uint8_t inq[NUM_IDS];  // Is PID in queue?
        uint8_t req[NUM_IDS];  // LOCK_SH or LOCK_EX, indexed by PID
    } waitq;
} flock_t;
```

Listing 1: flock_t structure (`kern/flock/flock.h`)

5.2 Writer Priority Strategy

To prevent writer starvation under high reader load, the engine implements a writer-priority policy. When one or more exclusive (write) lock requests are queued, new shared (read) lock requests are deferred until all pending writers have proceeded. This prevents an unbounded stream of readers from indefinitely blocking a waiting writer.

```
// Writer-priority: block new shared requests if exclusive waiters exist
if (fl->state == FLOCK_EXCLUSIVE || fl->waiting_exclusive > 0) {
    if (nonblocking) return -1;
    fl->waiting_shared++;
    while (fl->state == FLOCK_EXCLUSIVE || fl->waiting_exclusive > 0)
        thread_sleep(fl, &fl->lock);
    fl->waiting_shared--;
}
```

Listing 2: Writer-priority check inside `try_acquire_shared()` (`kern/flock/flock.c`)

5.3 Upgrade and Downgrade Support

The `file_flock()` function supports transitions between lock modes:

- **Shared → Exclusive (Upgrade):** Release shared, then acquire exclusive. May block if other readers are active.
- **Exclusive → Shared (Downgrade):** Release exclusive, then acquire shared. Allows new readers to proceed immediately.

5.4 File-System Integration (Modified Files)

Four existing kernel files were modified to integrate the new flock subsystem:

File	Change Made
`kern/fs/inode.h`	Added `flock_t flock` to `struct inode`, anchoring lock state at the inode level so all file descriptors to the same file share one lock object.
`kern/fs/inode.c`	Calls `flock_init(&ip->flock)` in `inode_get` when reusing a cached inode entry, ensuring lock state is always clean.
`kern/fs/file.h`	Added `holding_flock` to `struct file`, tracking the lock type currently held by this descriptor.
`kern/fs/file.c`	`file_alloc`: initializes `holding_flock = 0`. `file_close`: auto-releases any held lock before releasing the inode. `file_flock`: full lock logic including unlock, upgrade/downgrade, and engine calls.

Table 3: File-system integration changes

Auto-release on close ensures no lock is ever leaked:

```
void file_close(struct file *f) {
    // ...
    if (ff.type == FD_INODE) {
        if (ff.holding_flock != 0)
            flock_release(&ff.ip->flock, ff.holding_flock);
        begin_trans();
        inode_put(ff.ip);
        commit_trans();
    }
}
```

Listing 3: Auto-release logic in `file_close` (`kern/fs/file.c`)

5.5 Syscall Plumbing (Modified Files)

A complete end-to-end syscall was wired up from user space to the kernel:

File	Change
`kern/lib/syscall.h`	Added syscall number `SYS_flock`.
`kern/trap/TDispatch/TDispatch.c`	Added `case SYS_flock` in dispatcher.
`kern/fs/sysfile.c`	Implemented `sys_flock(tf_t *tf)`: fd validation, call to `file_flock`, return value and errno.
`kern/fs/sysfile.h`	Added declaration for `sys_flock`.
`user/include/syscall.h`	Added inline user wrapper `sys_flock(int fd, int operation)`.
`user/include/file.h`	Added `LOCK_SH`, `LOCK_EX`, `LOCK_UN`, `LOCK_NB` constants and `flock` macro.

Table 4: Syscall plumbing changes

5.6 Test Binaries (New Files)

Binary	Purpose
`user/flocktest/flocktest.c`	Main test suite. Runs 16 tests and prints `[PASS]` / `[FAIL]` per test with a final summary.
`user/flockchild/flockchild.c`	Persistent helper process for multi-process tests. Reads mode commands from `/flockchild.mode` and writes result codes to `/flockchild.res` using nonces to avoid stale results.

Table 5: Test binaries

6. System Architecture

6.1 Layered Syscall Pipeline

The design follows a strict layered architecture to keep each component's responsibility clear.

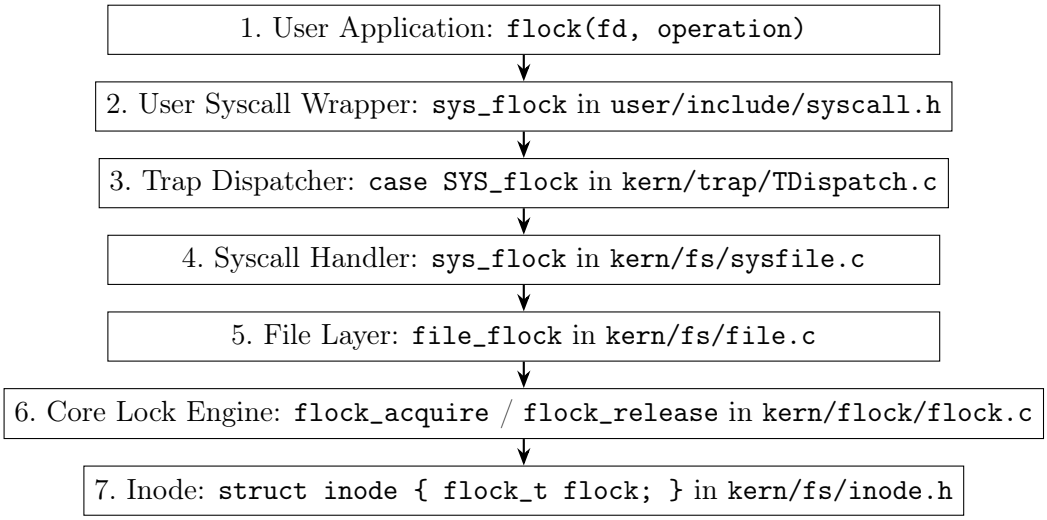

Figure 1: End-to-end call path for `flock(fd, op)`

6.2 Call Flow Description

1. The user application calls the `flock(fd, op)` macro defined in `user/include/file.h`.
2. The macro expands to the inline syscall wrapper `sys_flock`, which triggers a trap into the kernel.
3. The trap dispatcher in `TDispatch.c` routes the `SYS_flock` number to its handler.
4. The syscall handler in `sysfile.c` validates the file descriptor and operation flags.
5. The file layer in `file.c` applies per-descriptor ownership rules (upgrade, downgrade, unlock).

6. The core lock engine in `flock.c` enforces lock state, contention policy, and blocking behavior.
7. Lock state is stored directly in the inode so all descriptors sharing the same file coordinate through one object.

6.3 Per-Inode Lock State

Lock state is embedded directly into the inode structure:

```
struct inode {
    // ... existing fields ...
    flock_t flock;   // per-file lock state
};
```

Listing 4: Lock field added to inode structure (`kern/fs/inode.h`)

By anchoring the lock to the inode, all file descriptors referencing the same underlying file automatically share a single synchronization object. This is the correct semantic for `flock`: the lock is a property of the file (inode), not of the open file description or descriptor.

6.4 Per-Descriptor Ownership Tracking

Each file descriptor records what lock it currently holds:

```
struct file {
    // ... existing fields ...
    int holding_flock;  // FLOCK_INACTIVE, FLOCK_SHARED, or
   FLOCK_EXCLUSIVE
};
```

Listing 5: Ownership field added to file structure (`kern/fs/file.h`)

This field enables three important behaviors:

- **Consistent unlock:** the kernel knows what to release without requiring the caller to remember.
- **Upgrade/downgrade:** the file layer can release the old lock mode before acquiring the new one.
- **Clean resource release on close** (see Section 6.5).

6.5 Automatic Cleanup on Close

The `file_close` path checks `holding_flock` before releasing the inode, guaranteeing that no lock is ever leaked even when a process exits without explicitly calling `LOCK_UN`.

6.6 Multi-Process Test Coordination Protocol

Tests 12–16 require two processes to synchronize. Since mCertiKOS lacks System V shared memory or POSIX pipes in the usual form, a filesystem-based protocol was designed:

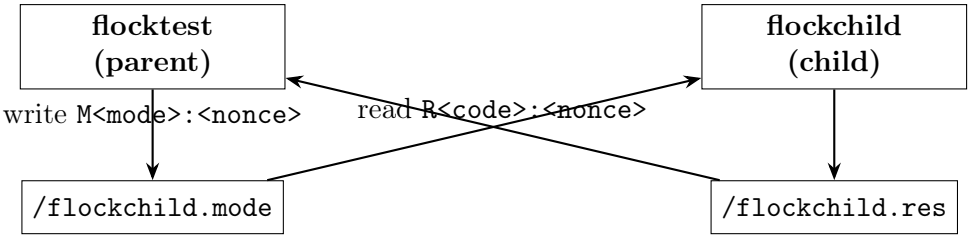

Figure 2: Parent–child communication protocol for multi-process tests

Each command and response is tagged with a unique nonce. The parent verifies that the nonce in the response matches the one it sent, preventing it from acting on a stale result from a previous test round.

7. Technologies, Frameworks, and Libraries Used

Technology	Role in Project
mCertiKOS	Target OS kernel. Provides the base file system, process model, and synchronization primitives.
C (C99)	All kernel and user-space code. GCC extensions and custom headers for OS-level types.
x86 Assembly	Syscall entry via `int $0x30`; trap frame manipulation for passing arguments and return values.
GCC (cross-compiler)	Compiles kernel and user binaries targeting the 32-bit x86 architecture used by mCertiKOS.
GNU Make	Build system. `Makefile.inc` files per module compose into the top-level `Makefile`.
QEMU (x86_64)	Hardware emulator used to boot mCertiKOS and run the test suite without physical hardware.
Spinlocks	Kernel primitive used inside `flock_t` to protect lock state counters and the wait queue.
`thread_sleep` / `thread_wakeup`	Kernel scheduler primitives for blocking lock acquisition, sleeping on the `flock_t` address as the wait channel.

Table 6: Technologies used

8. How Each Module Enhances the Project

8.1 `kern/flock/` — Core Lock Engine

Before this module existed, mCertiKOS had no notion of file locking. The core engine introduces the complete state machine (INACTIVE, SHARED, EXCLUSIVE) along with all the concurrency logic: atomic state transitions under a spinlock, writer-priority queuing to prevent starvation, and the blocking infrastructure (sleep/wakeup) for processes that must wait. Without this module, none of the higher layers have anything to call.

8.2 Inode-Level `flock_t` — Correct Scope for Lock State

By embedding `flock_t` in the inode, the project ensures that locking is file-scoped rather than descriptor-scoped. Multiple processes opening the same file automatically participate in the same lock, which is the semantically correct behavior specified by POSIX `flock(2)`.

8.3 File Descriptor `holding_flock` — Safe Resource Management

The `holding_flock` field on each file descriptor enables automatic, leak-free lock management. The kernel can always determine what a descriptor holds and release it exactly once, whether on an explicit `LOCK_UN` call or on fd close. This eliminates an entire class of resource-leak bugs without requiring any user-space discipline.

8.4 Syscall Plumbing — Usable from User Space

Without the syscall layer, the lock engine would exist but be inaccessible from user programs. The syscall plumbing bridges the user-kernel boundary, validates inputs defensively (bad fd, out-of-range fd), and propagates errors as standard errno values, making the feature production-quality.

8.5 `flocktest` and `flockchild` — Confidence in Correctness

A kernel feature that cannot be tested is only assumed to work. The 16-test suite provides systematic coverage across all code paths: basic operations, conflict detection, upgrade/downgrade, non-blocking mode, automatic cleanup, invalid input, and true multi-process blocking scenarios. The multi-process tests in particular exercise the sleep/wakeup machinery under real inter-process scheduling, which is impossible to verify with single-process tests alone.

9. Testing and Validation

9.1 Test Suite Overview

A total of **16 tests** are run by invoking `flocktest` from the mCertiKOS shell. Each test prints `[PASS]` or `[FAIL]`, followed by a final summary.

#	Test Name	What is Verified
1	**Basic Shared Lock**	`LOCK_SH` acquire and release basic path.
2	**Basic Exclusive Lock**	`LOCK_EX` acquire and release basic path.
3	**Multiple Shared Locks**	Two descriptors hold shared lock simultaneously (reader concurrency).
4	**Exclusive Blocks Shared (NB)**	`LOCK_EX` holder causes `LOCK_SH|NB` to fail; succeeds after unlock.
5	**Shared Blocks Exclusive (NB)**	`LOCK_SH` holder causes `LOCK_EX|NB` to fail; succeeds after unlock.
6	**Lock Upgrade (SH → EX)**	Same descriptor escalates from shared to exclusive.
7	**Lock Downgrade (EX → SH)**	Same descriptor de-escalates from exclusive to shared.
8	**Auto-release on Close**	Closing a locked fd releases lock; next fd can lock immediately.
9	**Invalid FD Handling**	`flock(-1, ...)` and `flock(999, ...)` return failure safely.
10	**Write with Exclusive Lock**	Write operation succeeds correctly while holding `LOCK_EX`.
11	**Read with Shared Lock**	Read operation succeeds correctly while holding `LOCK_SH`.
12	**Multi-process EX Blocks Child (NB)**	Parent's `LOCK_EX` blocks both `SH|NB` and `EX|NB` from child.
13	**Multi-process SH Blocks EX (NB)**	Parent's `LOCK_SH` blocks child's `LOCK_EX|NB` attempt.
14	**Child Succeeds After Release (NB)**	Child's `LOCK_EX|NB` succeeds after parent fully unlocks.
15	**Blocking EX Waits for SH Release**	Child's blocking `LOCK_EX` sleeps until parent releases `LOCK_SH`.
16	**Blocking SH Waits for EX Release**	Child's blocking `LOCK_SH` sleeps until parent releases `LOCK_EX`.

Table 7: The 16 validation tests

9.2 Test Category Breakdown

Category	Tests	Count
Basic lock operations	1, 2, 3	3
Conflict detection (non-blocking)	4, 5	2
Upgrade / downgrade	6, 7	2
Auto-cleanup and error handling	8, 9	2
I/O under lock	10, 11	2
Multi-process non-blocking	12, 13, 14	3
Multi-process blocking (sleep/wake)	15, 16	2
Total		**16**

Table 8: Tests grouped by category

9.3 How to Run

```
# Step 1: Build the kernel and user programs
make

# Step 2: Launch mCertiKOS in QEMU
make qemu

# Step 3: Inside the mCertiKOS shell, run the test suite
flocktest
```

Listing 6: Build and run the test suite

Expected output:

```
Test 1:  Basic Shared Lock           [PASS]
Test 2:  Basic Exclusive Lock        [PASS]
...
Test 16: Blocking Shared Waits       [PASS]

Summary: 16/16 tests passed.
```

10. Problems Encountered and How They Were Solved

10.1 Consistent Coordination Across Multiple File Descriptors

Challenge: When the same file is opened through different file descriptors (even by different processes), they must coordinate through a single lock state — not through separate per-descriptor states. Without this, two processes could simultaneously hold an exclusive lock.

Solution: Lock state (`flock_t`) is stored in the *inode*, not the file descriptor or file description. Since all opens of the same file share an inode, they automatically share one

lock object. This required modifying `struct inode` and initializing the flock on inode reuse in `inode_get`.

10.2 Avoiding Lock Leaks

Challenge: If a process holds a lock and exits or closes the file descriptor without calling `LOCK_UN`, the lock would remain permanently held, deadlocking any future accessor.

Solution: The `file_close` path always checks `holding_flock` and calls `flock_release` before releasing the inode reference. This is unconditional and happens at the kernel level, so no user-space cooperation is required.

10.3 Writer Starvation Under High Reader Load

Challenge: In a naive implementation, a continuous stream of new readers could prevent an exclusive waiter from ever acquiring the lock, as each new reader arrives while others are still active, keeping `shared_holders` above zero indefinitely.

Solution: The writer-priority strategy delays new shared lock requests when exclusive waiters are present (`waiting_exclusive > 0`). New shared requests sleep rather than being admitted immediately, giving the exclusive waiter a guaranteed window to proceed once current readers finish.

10.4 Multi-Process Test Coordination Without Shared Memory

Challenge: mCertiKOS does not provide System V shared memory or pipes in the same way a full UNIX OS does, making it non-trivial to synchronize a parent and child test process reliably.

Solution: A filesystem-based protocol using two files (`/flockchild.mode` and `/flockchild.res`) with nonce-tagged messages provides reliable, stale-free coordination between `flocktest` and the `flockchild` helper process. The parent verifies the nonce in each response before acting on it.

10.5 Ensuring Inode Lock State Is Never Stale

Challenge: The kernel inode cache reuses inode slots. If a file is deleted and the slot is reused for a new file, the old lock state from the previous file's users could contaminate the new file.

Solution: A call to `flock_init(&ip->flock)` was added in `inode_get` at the point where a cached inode entry is recycled. This guarantees a clean, `FLOCK_INACTIVE` state for every new inode usage.

11. Learning Outcomes

Through this project, the team gained practical, hands-on understanding of:

- **Kernel synchronization primitives:** spinlocks, wait queues, `thread_sleep`, and `thread_wakeup` as building blocks for higher-level lock semantics.

- **Lock state machines:** modeling the transitions between inactive, shared, and exclusive states and handling contention correctly in each state.
- **Syscall interface design:** adding a new syscall end-to-end — from the user-space macro through the trap table to the kernel handler and back.
- **File-system integration:** embedding new state into kernel data structures (inode, file) and hooking into existing life-cycle events (`inode_get`, `file_close`).
- **Concurrent test design:** designing multi-process test scenarios that reliably detect concurrency bugs without relying on timing assumptions.

12. Conclusion

The FLOCK project successfully integrates robust BSD-style file locking into mCertiKOS. The implementation is architecturally clean, following the kernel's existing layered design, and semantically complete, supporting all four standard operations with both blocking and non-blocking modes.

The 16-test validation suite provides strong confidence across a wide range of scenarios: basic operations, conflict detection, upgrade/downgrade paths, automatic cleanup, invalid input handling, and true multi-process concurrency with blocking sleep/wakeup behavior.

The project demonstrates that a non-trivial kernel feature — one involving concurrency, resource lifetime management, and cross-process coordination — can be added incrementally and validated systematically in a teaching OS environment, reinforcing both the theory and practice of operating systems design.

A. File Inventory

File Path	**Role**
`kern/flock/flock.h`	Lock state types and constants (NEW)
`kern/flock/flock.c`	Core lock engine: acquire, release, init (NEW)
`kern/flock/export.h`	Public API for other kernel modules (NEW)
`kern/fs/inode.h`	Added `flock_t flock` to inode (MODIFIED)
`kern/fs/inode.c`	Calls `flock_init` on inode reuse (MODIFIED)
`kern/fs/file.h`	Added `holding_flock` to file descriptor (MODIFIED)
`kern/fs/file.c`	`file_alloc`, `file_close`, `file_flock` (MODIFIED)
`kern/lib/syscall.h`	`SYS_flock` number (MODIFIED)
`kern/trap/TDispatch/TDispatch.c`	Syscall dispatch case (MODIFIED)
`kern/fs/sysfile.c`	`sys_flock` handler (MODIFIED)
`kern/fs/sysfile.h`	`sys_flock` declaration (MODIFIED)
`user/include/syscall.h`	User-space syscall wrapper (MODIFIED)
`user/include/file.h`	Lock flags and `flock` macro (MODIFIED)
`user/flocktest/flocktest.c`	16-test validation suite (NEW)
`user/flockchild/flockchild.c`	Multi-process helper child (NEW)